

Cluster-GCN

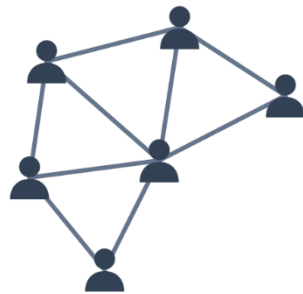
An Efficient Algorithm for Training
Deep and Large Graph Convolutional Networks

Wei-Lin Chiang · Xuanqing Liu · Si Si · Yang Li · Samy Bengio · Cho-Jui Hsieh

National Taiwan University · UCLA · Google Research | KDD '19

Graph Data와 GNN : 이웃을 보고 나를 정의한다

현실의 많은 데이터는 graph data로 표현



SNS 친구 관계

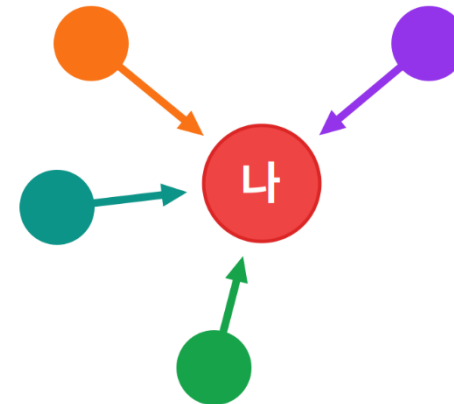
node = 유저
edge = 팔로우 관계



함께 구매한 상품

node = 유저, 상품
edge = 구매한 상품

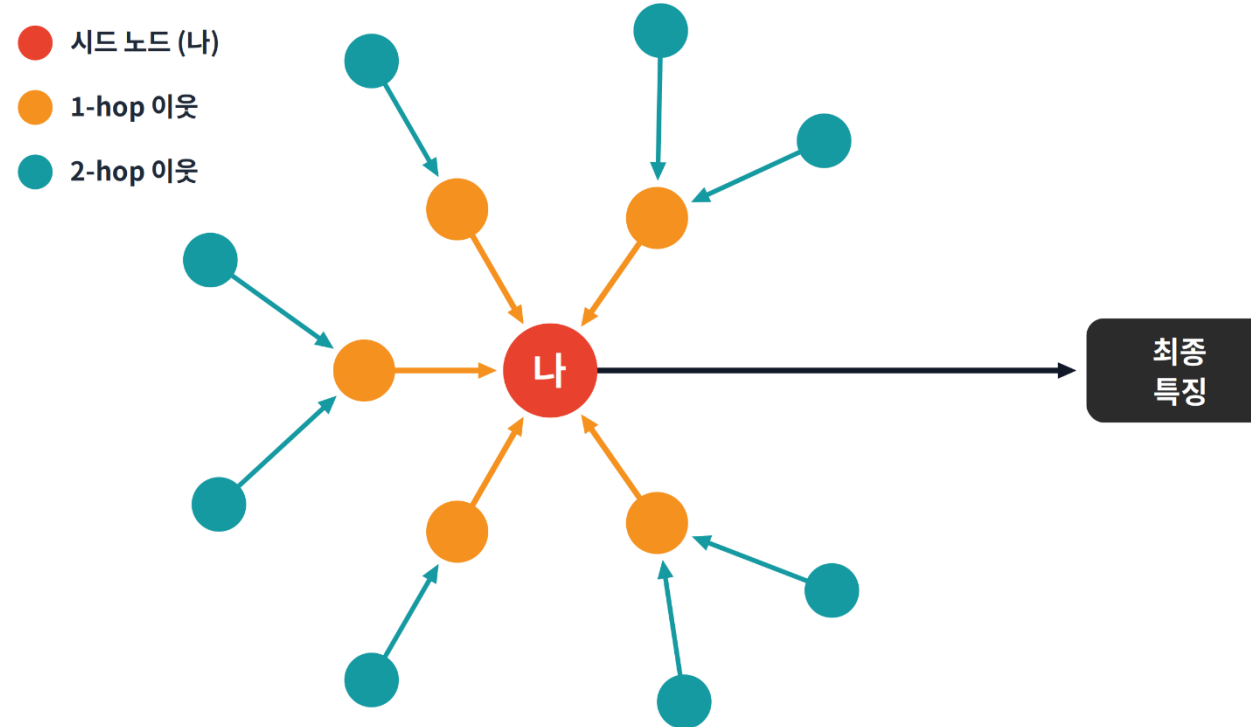
GNN의 핵심 원리



"내 주변 이웃을 종합해 나를 정의한다"

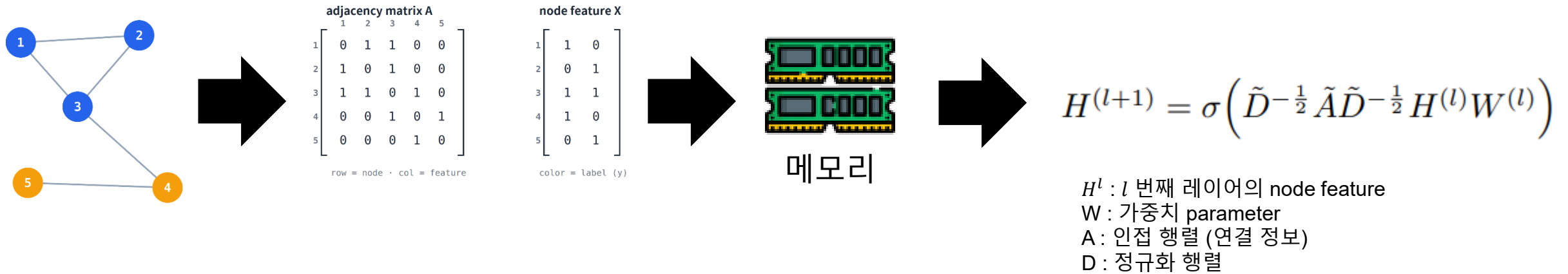
- 현실의 많은 데이터는 node와 edge로 이루어진 graph data
ex) SNS 친구 관계, 온라인 쇼핑 상품 구매 등
- GNN은 이러한 그래프를 학습하고 예측하는 가장 대표적인 방법.
- 핵심 원리 : "내 주변 이웃들의 정보를 종합해 나의 정체성(특징)을 결정한다." = **Message Passing**

Message Passing : 이웃의 정보를 한 홉씩 집계한다



- 각 노드는 자신과 연결된 이웃의 특징을 모아(aggregate) 자신의 feature 을 갱신.
- 멀리 떨어진 이웃 → 가까운 이웃 → 나 순으로, 정보가 한 hop씩 안쪽으로 전달(passing), 집계(aggregate).
- layer을 여러 개 쌓을수록 더 먼 2-hop, 3-hop 이웃의 정보까지 반영.

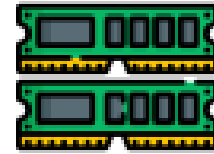
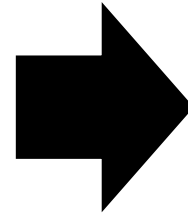
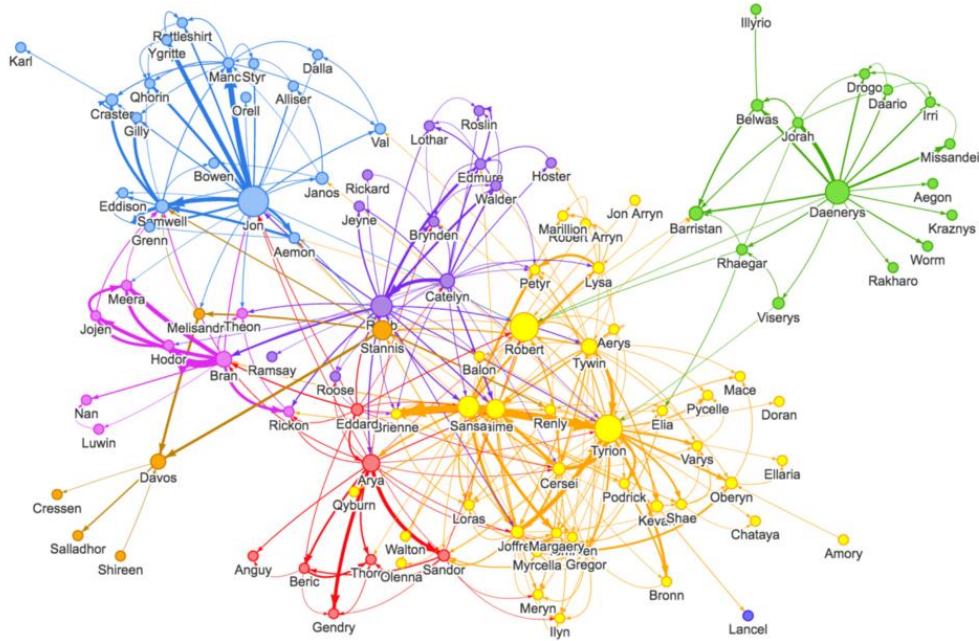
GNN의 구현 : Full-batch GCN



- GNN의 학습 방법을 수학적으로 정의.
- 전통적인 Full batch GCN은 그래프의 모든 정보를 메모리에 올림
- 모든 노드는 자신의 이웃 정보를 모으고 집계하여 feature를 업데이트.

Reference: Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. ICLR 2017.

전통적인 방식의 GCN의 한계

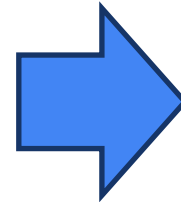
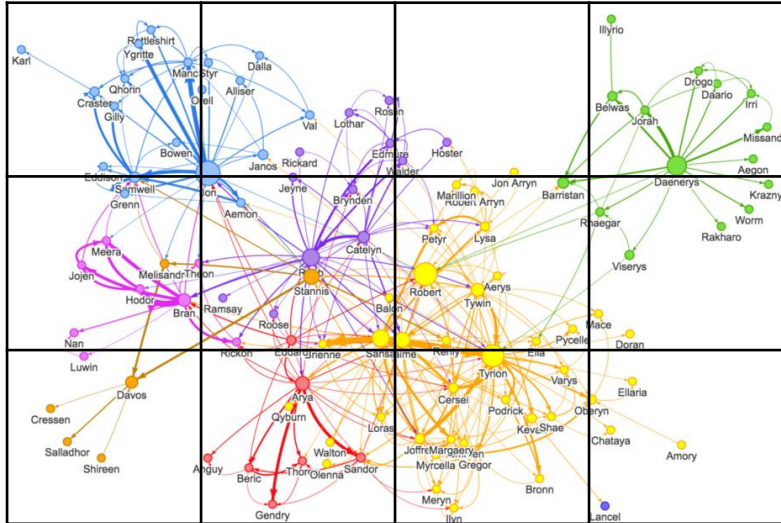


Out Of Memory!

거대해지는 그래프 데이터의 규모

- GCN은 node feature와 연결 정보(Adjacency Matrix)를 모두 메모리에 올려야 함
- 하지만 시간이 지나며 그래프 크기가 커짐에 따라 메모리에 모두 못 올리는 경우가 발생
- 따라서 그래프의 일부만 메모리에 올려 처리하는 Minibatch SGD 방식의 필요성이 대두.

그래프의 일부만 올리는 방식의 연구 필요성



- Example: CiteSeer (a citation network with 3327 nodes)
- If applying **graph partitioning** (e.g., metis) on **A** and removed between-cluster edges, the accuracy of GCN **remains similar** (even though **20% edges are removed**)

CiteSeer	Random Partitioning	Graph Partitioning
1 (no partitioning)	72.0	72.0
100 partitions	46.1	71.5 (~20% edges cut)

단순히 랜덤하게 자르면? => 잘린 부분의 edge 정보 손실

성능 하락으로 이어져버림

- CNN처럼 그래프를 단순히 나누면 나뉘는 부분의 edge가 사라져 정보 손실 발생
- 이 정보 손실로 인해 정확도가 크게 떨어짐
- 따라서 그래프의 일부만 올리되 정보 손실을 최소화되게끔 하는 Graph SAGE, Fast GCN 과 같은 똑똑한 partitioning 방식이 제안됨

Reference: Chiang et al. (2019). Cluster-GCN, KDD 2019. Poster: <https://infwinston.github.io/files/kdd19/poster.pdf>

Minibatch 방법1. GraphSAGE – 노드 피처를 output으로 하는 함수 학습

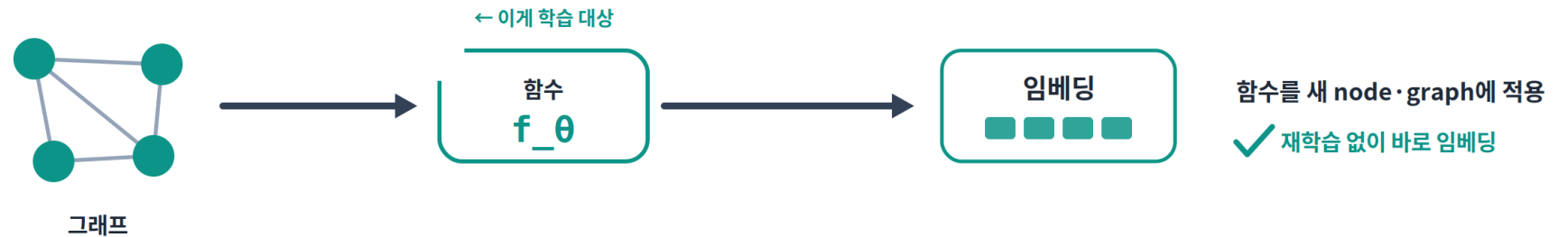
기존 방식

Transductive
(GCN 등)



GraphSAGE

Inductive

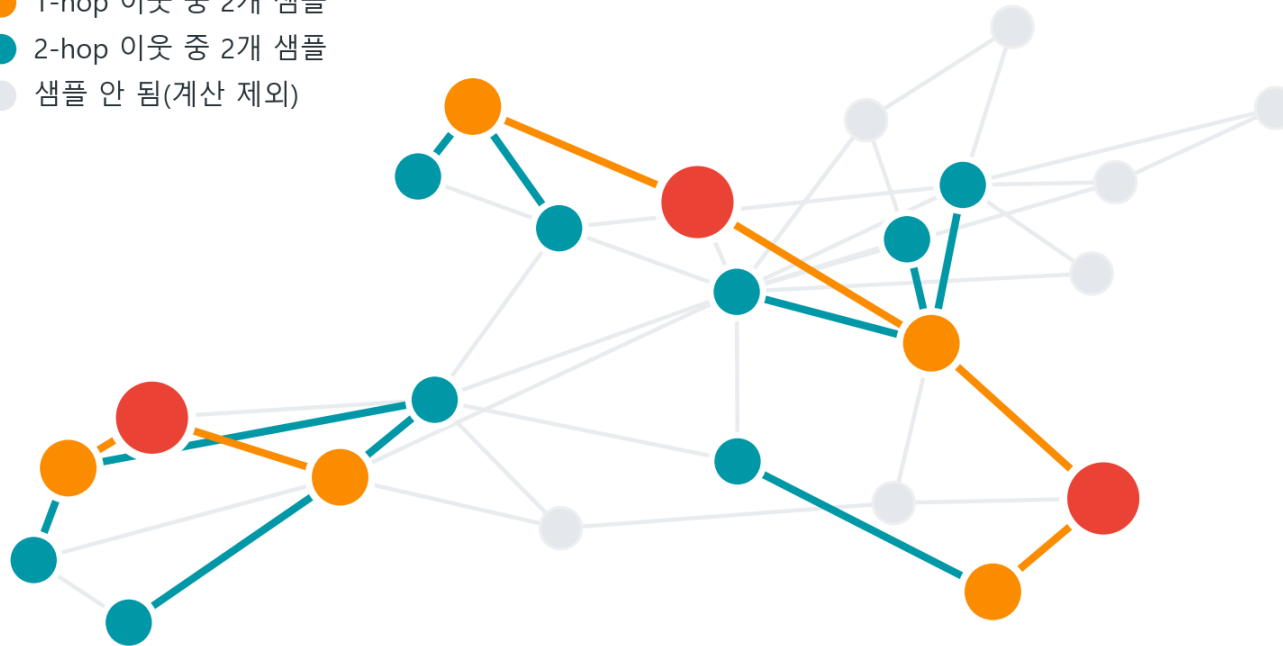


- Graph SAGE는 그래프 분석에서 새 노드가 와도 바로 inference가 가능한 inductive한 학습 방식을 만드는 것이 목표
- 이를 위해 기존 그래프를 학습하여 바로 임베딩을 만드는 것에서 벗어나 임베딩을 만드는 **함수를 학습시킴**
- 기존에는 새 노드가 들어오면 재학습이 필요했지만 SAGE는 학습된 함수에 넣으면 바로 임베딩 결과를 얻을 수 있음

Reference: Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. NeurIPS 2017.

GraphSAGE : 이웃을 '고정 개수'만 샘플링

- 시드 노드 (무작위 선택)
- 1-hop 이웃 중 2개 샘플
- 2-hop 이웃 중 2개 샘플
- 샘플 안 됨(계산 제외)



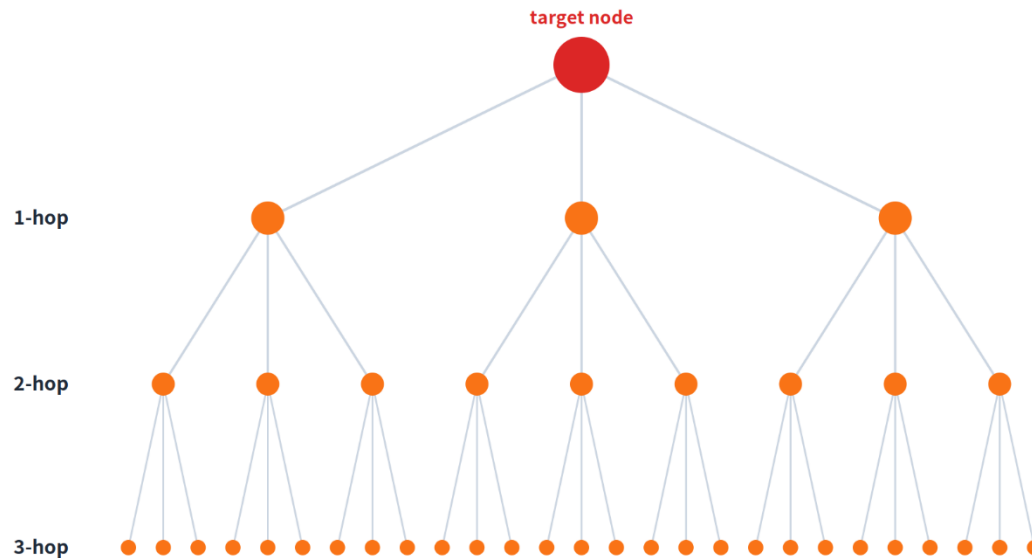
각 노드마다 이웃을 '고정 개수(예: 2개)'만 샘플링 → 이웃이 겹치지 않는 최악의 경우에도 메모리 상한이 고정

- Neighborhood Sampling : Mini-batch의 시드 노드들의 이웃 중, 정해진 수의 이웃만을 무작위로 추출하여 메모리에 올리는 방식
- 따라서 그래프 전체를 메모리에 올리는 대신 시드 노드와 그 시드 노드의 고정된 수의 일부 이웃들만 올려 메모리 문제 해결

※ GraphSAGE에서는 1-hop, 2-hop을 각각 25, 10 개를 샘플링

Minibatch 방법2. FastGCN – 이웃 폭발을 해결하여 레이어를 더 많이 쌓도록

node별로 이웃을 20개씩 재귀적으로 뽑으면, 층마다 $\times 20 \rightarrow$ 참조 노드가 지수적으로 증가



층별 참조 노드 수

1-hop : 20

2-hop : $20 \times 20 = 400$

3-hop : $20 \times 20 \times 20 = 8,000$

4-hop : $20 \times 20 \times 20 \times 20 = 160,000$

...

L-hop : 20^L

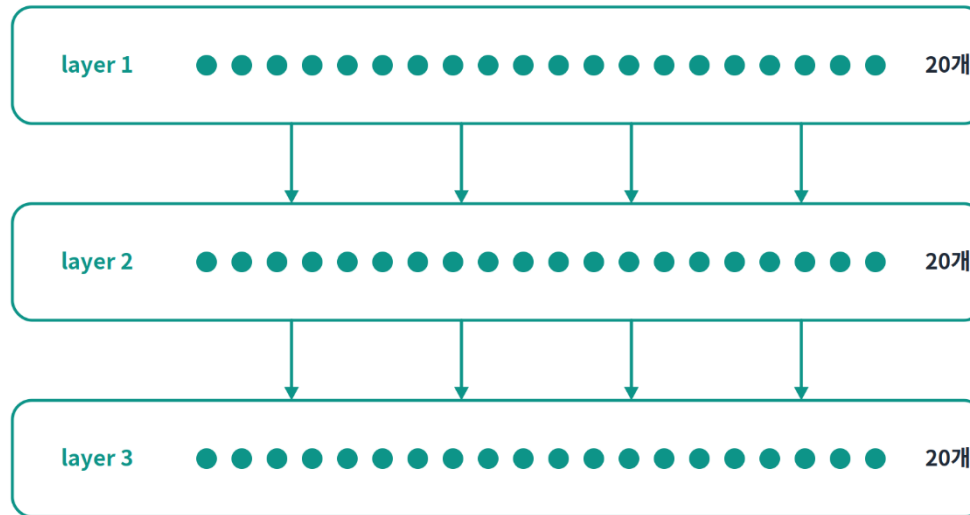
→ 지수적 폭발

- 레이어를 2층, 3층 쌓으면 2-hop, 3-hop의 이웃까지 샘플링 함. 즉, 레이어를 깊게 쌓을수록 더 멀리 있는 이웃을 참조.
- 그런데 레이어 층을 쌓을 때마다 참조해야 하는 노드가 지수적으로 증가 => **Neighborhood Explosion!**
- 이로 인해 레이어 층을 깊게 쌓지 못하는데, FastGCN은 이를 해결하고자 함.

Reference: Chen, J., Ma, T., & Xiao, C. (2018). FastGCN: Fast Learning with Graph Convolutional Networks via Importance Sampling. ICLR 2018.

FastGCN : 레이어당 고정된 수의 이웃만을 샘플링

각 층에서 전체 노드로부터 독립적으로 20개만 뽑음 → 총 표본 = $20 \times L$



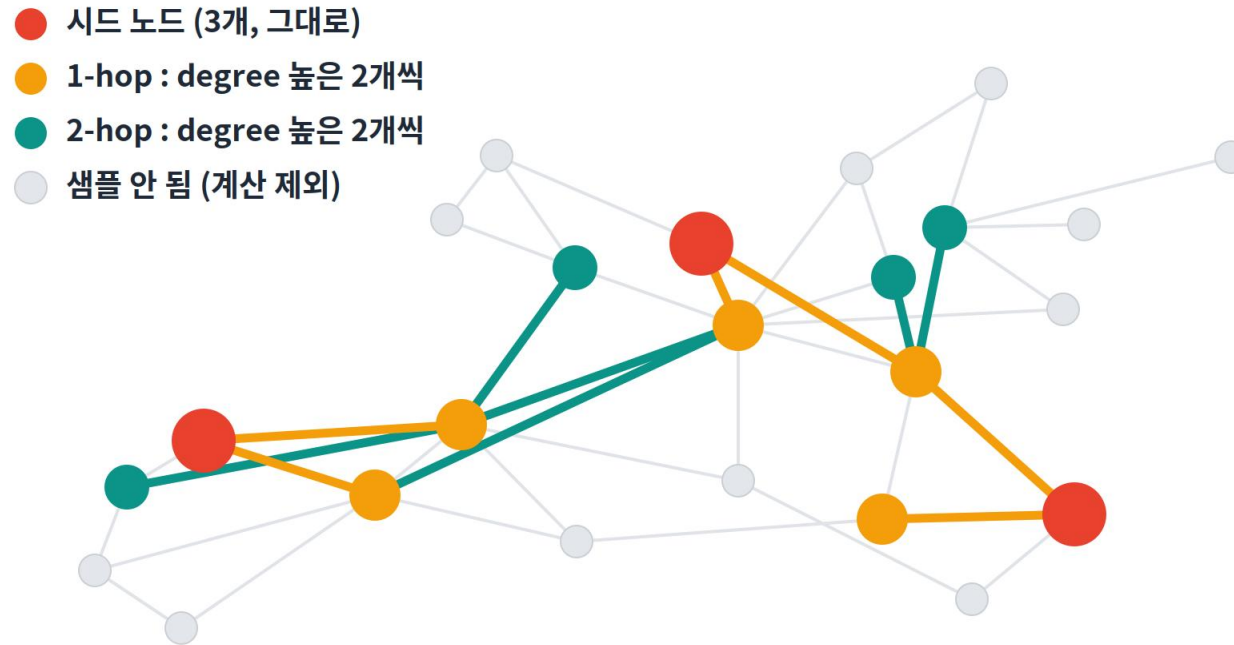
hop별 누적 표본 수

1-hop : 20
 2-hop : $20+20 = 40$
 3-hop : $20+20+20 = 60$
 4-hop : $20+20+20+20 = 80$
 ...
 L-hop : $20 \times L$

→ 선형 증가

- 하나의 노드당 고정된 수의 이웃이 아닌 하나의 레이어당 고정된 수의 이웃을 뽑음
- 레이어를 하나 쌓을 때마다 20개만 추가로 보면 됨 => 지수적 증가에서 선형적 증가로.

레이어당 중요한 노드 위주로 샘플링



각 seed의 1-hop·2-hop에서 degree 높은 2개씩만 샘플링 → 계산에 쓰이는 노드를 고정·축소

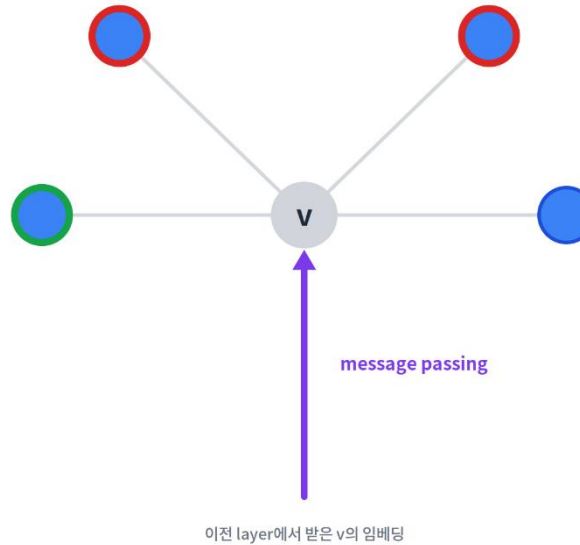
- 무작위로 샘플링 하는 대신 노드의 중요도에 따라 확률적으로 샘플링 → 최대한 정보손실 최소화
- 중요한 노드(degree가 높은 노드)에 높은 샘플링 확률 부여
- 즉, 영향력이 큰 일부 노드만 메모리에 올려 샘플링을 적게하며 성능이 떨어지는 영향을 최소화.

Mini-batch SGD의 특징 : 좋은 확장성, 하지만 epoch당 시간은 증가

- 그래프의 일부만 올리는 Mini-batch SGD방식은 확장성(메모리 관리) 면에서는 매우 유리함
- 하지만 GCN과 비교해서 계산량이 증가하기 때문에 epoch당 시간은 증가함 = Embedding Utilization이 낮다!

Embedding Utilization : 임베딩을 몇 개의 노드가 재사용하나

- 빨간색 테두리 : 배치 1의 시드 노드
- 초록색 테두리 : 배치 2의 시드 노드
- 파란색 채움 : Full-batch GCN에서 v의 임베딩을 사용하는 노드



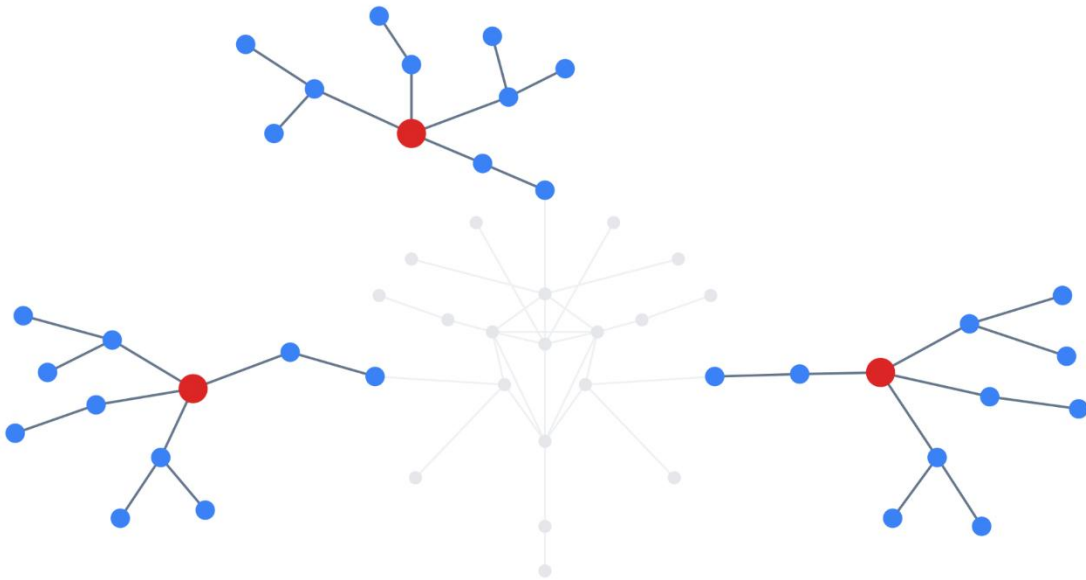
Embedding Utilization 값

● Mini-batch SGD · 배치 1	2
● Mini-batch SGD · 배치 2	1
● Full-batch GCN	4 (= degree)

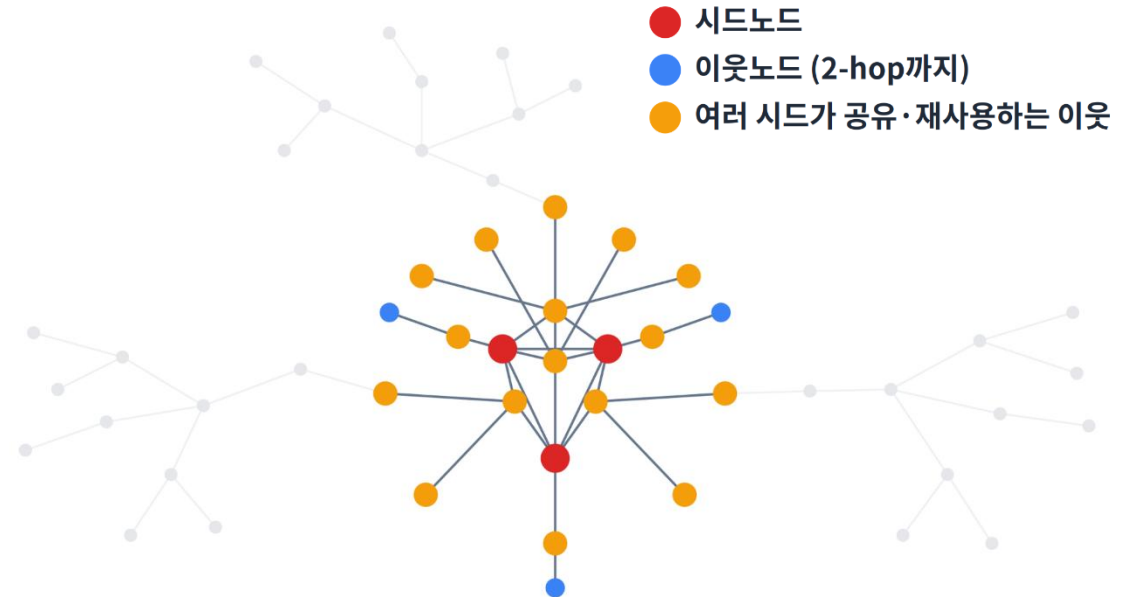
- Embedding utilization의 정의 : 이전 레이어에서 계산해둔 임베딩을 다음 레이어에서 몇 번 사용되었는가?
- Mini-batch : Minibatch1에서는 노드 v의 결과값을 3개의 노드가 사용하므로 Embedding Utilization = 3 (빨간색 테두리)
- Minibatch2 에서는 Embedding Utilization = 1 (초록색 테두리)
- Full GCN : 모든 노드가 학습에 참여하므로 그 노드의 모든 이웃이 재사용 → utilization = 이웃 수(최대).

Embedding Utilization이 낮으면 계산량이 많아진다.

Embedding Utilization이 낮은 경우



Embedding Utilization이 높은 경우



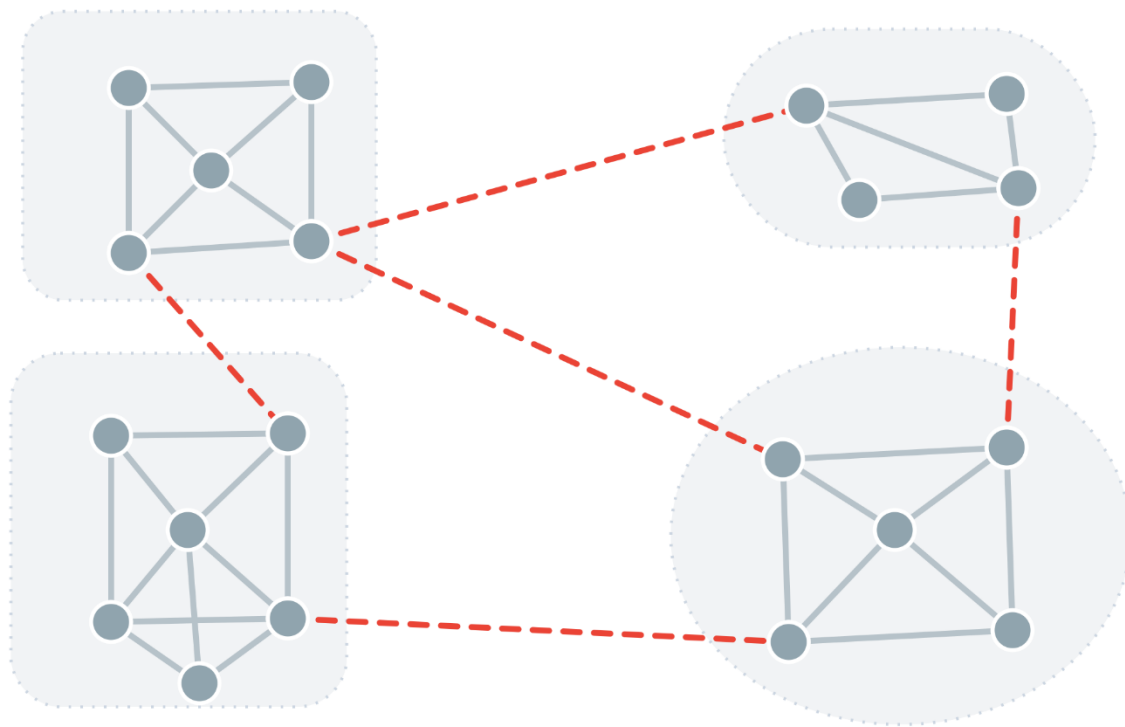
- Embedding Utilization이 낮으면 공유 이웃이 적기 때문에 각각의 계산을 모두 개별로 수행해야 함.
- 반대로 Embedding Utilization이 높으면 이전의 계산 결과를 여러 노드가 재활용할 수 있고 그만큼 연산량이 줄어듦.

공유하는 이웃이 많은 시드 노드들 위주로 뽑아야 하는 이유 정리

- Full batch GCN은 그래프 크기가 크면 메모리에 올릴 수 없어 일부만 올리는 Minibatch-SGD를 사용
- 하지만 Minibatch-SGD는 연산량이 많아져서 에포크당 시간이 더 오래걸림
- 이는 이전 레이어 값을 재사용하는 Embedding Utilization값이 낮아서 그럼.
- 따라서 연산량을 최대한 줄이기 위해서는 Embedding Utilization 값을 최대화해야 함.
- Embedding Utilization을 최대한 올리려면 가능한한 공유하는 이웃이 많도록 배치 내의 시드 노드들을 구성해야 함.

→ 하나의 배치에서 최대한 공유하는 이웃이 많도록 시드 노드를 뽑으면 epoch당 시간이 줄어든다!

그래프의 성질 : 노드는 '커뮤니티'를 이룬다

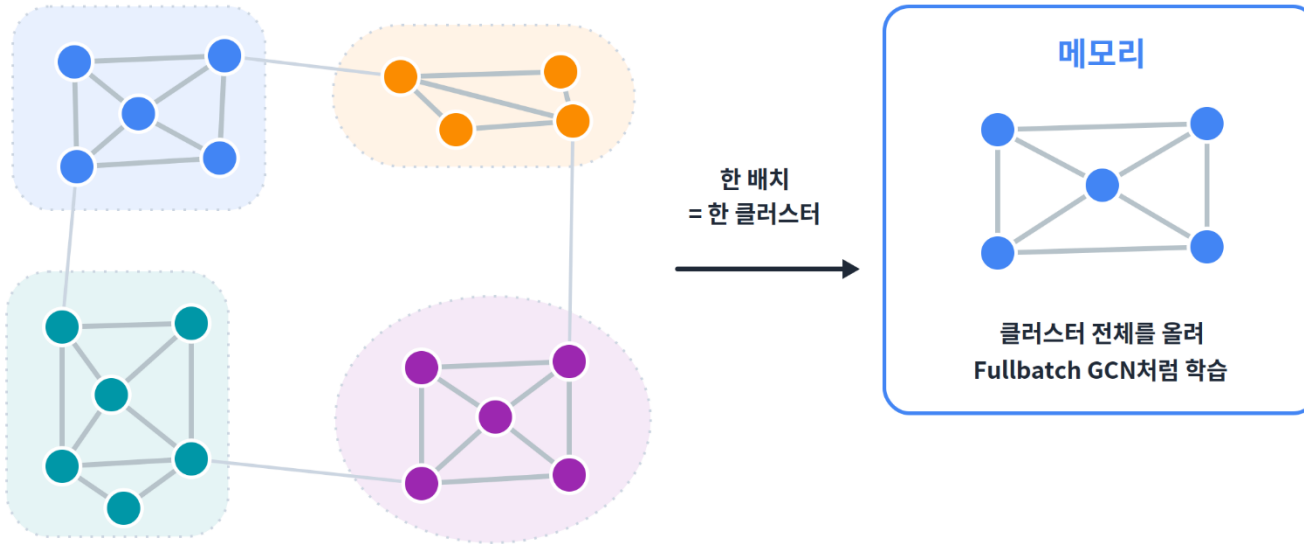


- 커뮤니티 내부 : 촘촘하게 연결 (공유 이웃 많음)
- - - 커뮤니티 사이 : 드물게 연결

- 그래프의 노드는 연결이 균등하지 않고 여러 커뮤니티를 형성
- 커뮤니티 내부는 연결이 촘촘하고, 커뮤니티 간에는 연결이 드물
- 따라서 시드 노드를 뽑을때 하나의 커뮤니티로 범위를 제한시키면 공유 이웃이 많아져 embedding utilization이 올라감.

Cluster-GCN의 아이디어 : 클러스터 안에서만 학습

METIS로 커뮤니티(클러스터) 단위 분할



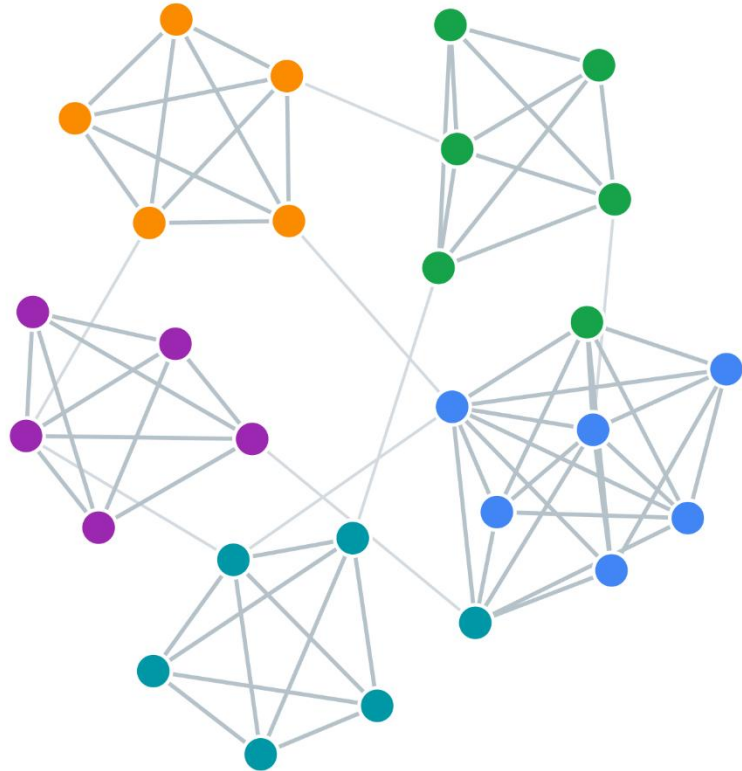
“하나의 Batch에 seed node 구성을 하나의 커뮤니티 내에서 하자!”

- 그래프 데이터를 학습 전, 먼저 클러스터링부터 실행하여 커뮤니티 단위로 그래프를 분할
- 하나의 Batch = 하나의 커뮤니티로만 구성
- 클러스터 내부는 연결이 촘촘함
→ Embedding Utilization이 극대화
→ 에포크당 시간 감소

Dataset	random partition	clustering partition
Cora	78.4	82.5
Pubmed	78.9	79.9
PPI	68.1	92.9

무작위 분할과 클러스터링 분할의 F1 score

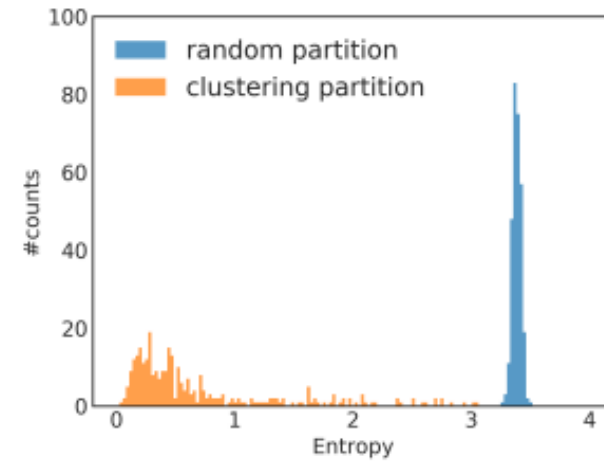
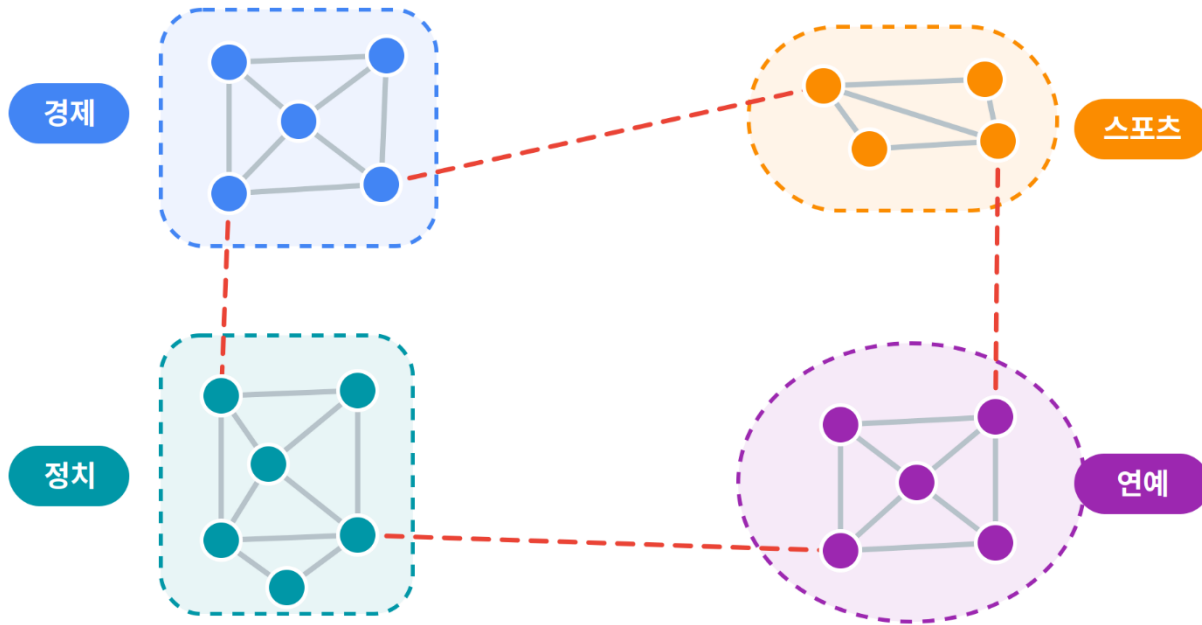
METIS : 클러스터간 크기를 최대한 동등하게



- METIS는 커뮤니티에 있는 노드 수를 최대한 동등하게 함
- 커뮤니티의 크기가 최대한 비슷하도록 나눔 이때 이점 2가지
 1. 매 Minibatch마다 생성되는 노드 feature의 행렬의 차원이 균일하여 병렬 처리 효율 극대화
 2. 배치의 크기가 제각각이면 너무 작을 경우는 GPU낭비, 너무 크면 OOM의 위험이 있는데 METIS는 이 위험이 없음

문제 : 배치가 한 클러스터에 편향된다

한 배치 = 한 커뮤니티 = 한 주제만 → 성향이 편향됨

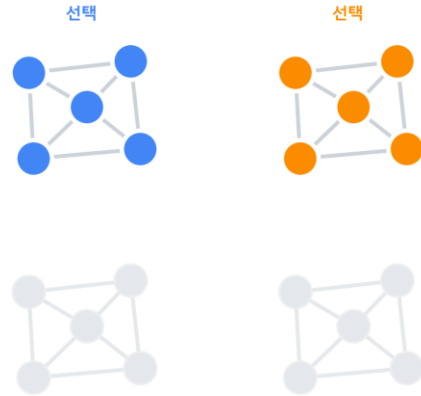


클러스터 배치는 라벨 엔트로피가 낮음 = 라벨이 편중됨

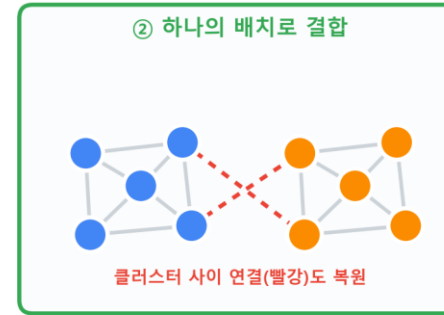
- 같은 클러스터의 노드들은 성향이 비슷하고, 다른 클러스터의 노드들과는 이질적
- 배치마다 한 클러스터(=한 성향)만 학습
→ 매 배치가 특정 성향에 치우친 노드로만 업데이트
- 라벨이 편중(낮은 엔트로피) → 배치 간 편차 ↑ → 수렴이 불안정

보완 : 여러 클러스터를 묶는 Stochastic Multi-Clustering

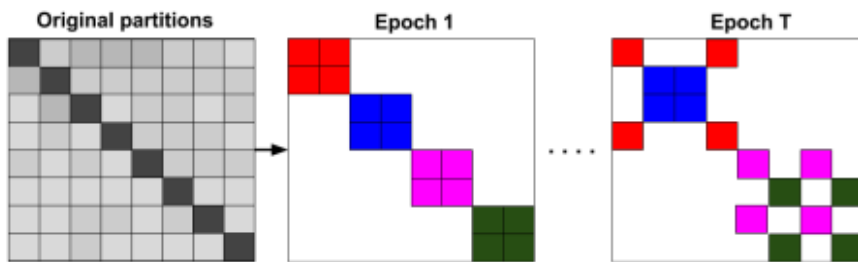
① 무작위로 q 개 클러스터 선택 ($q=2$)



② 하나의 배치로 결합



매 배치마다 다른 조합 → 배치 간 분산 감소 → 수렴이 빨라지고 안정



x축 / y축 : 두 클러스터간 연결관계

예1) 1행 2열 : 클러스터 1과 클러스터 2의 연결 정보

예2) 3행 3열 : 클러스터 3의 내부 연결 정보



여러 클러스터(파랑)가 더 안정적으로 수렴

- 배치당 클러스터 1개 대신 q 개를 무작위로 뽑아 하나의 배치로 결합.
- 제거됐던 클러스터 사이 연결도 복원.
- 배치 간 분산 ↓ → 수렴이 빨라지고 안정(파란 곡선).

실험 설정 : 데이터셋과 하이퍼파라미터

Table 3: Data statistics

Datasets	Task	#Nodes	#Edges	#Labels	#Features
PPI	multi-label	56,944	818,716	121	50
Reddit	multi-class	232,965	11,606,919	41	602
Amazon	multi-label	334,863	925,872	58	N/A
Amazon2M	multi-class	2,449,029	61,859,140	47	100

Table 4: The parameters used in the experiments.

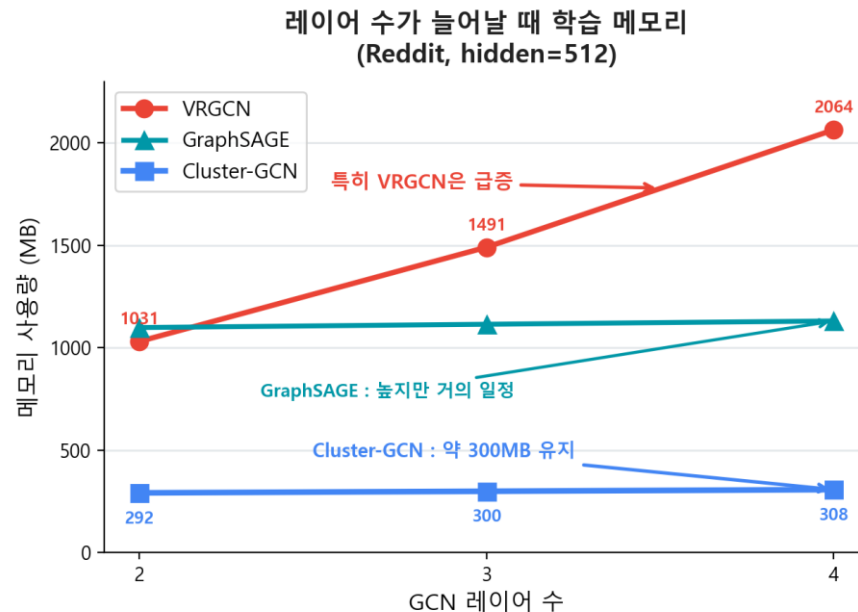
Datasets	#hidden units	# partitions	#clusters per batch
PPI	512	50	1
Reddit	128	1500	20
Amazon	128	200	1
Amazon2M	400	15000	10

- 4개 공개 데이터셋에서 multi-class / multi-label 노드 분류로 평가.
- PPI·Amazon = multi-label, Reddit·Amazon2M = multi-class.
- #partitions = 그래프를 몇 개 클러스터로 나눴는지.
- #clusters per batch = 한 배치에 묶는 클러스터 수(= q).

결과 ① 확장성 : 층이 깊어져도 메모리가 늘지 않는다

Table 5: Comparisons of memory usages on different datasets. Numbers in the brackets indicate the size of hidden units used in the model.

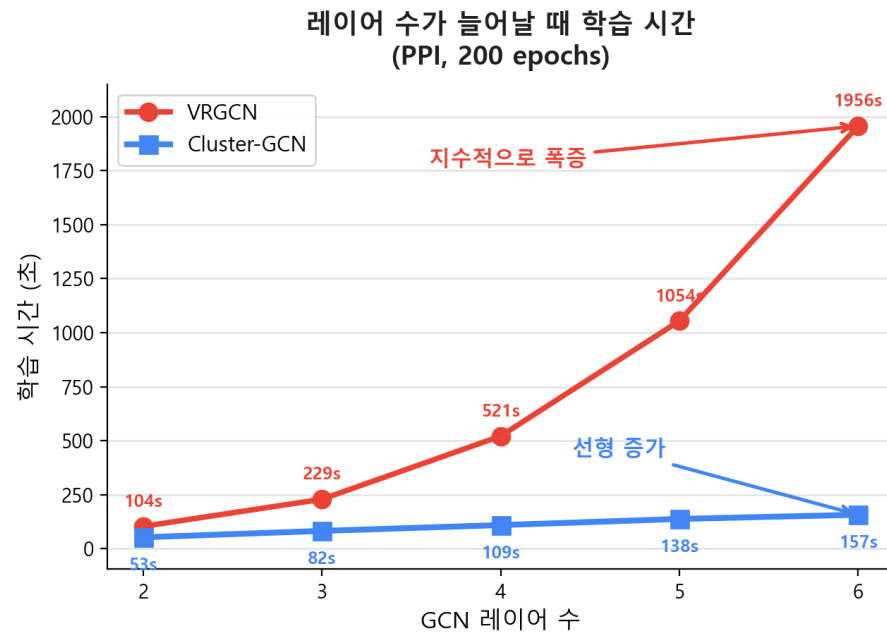
	2-layer			3-layer			4-layer		
	VRGCN	Cluster-GCN	GraphSAGE	VRGCN	Cluster-GCN	GraphSAGE	VRGCN	Cluster-GCN	GraphSAGE
PPI (512)	258 MB	39 MB	51 MB	373 MB	46 MB	71 MB	522 MB	55 MB	85 MB
Reddit (128)	259 MB	284 MB	1074 MB	372 MB	285 MB	1075 MB	515 MB	285 MB	1076 MB
Reddit (512)	1031 MB	292 MB	1099 MB	1491 MB	300 MB	1115 MB	2064 MB	308 MB	1131 MB
Amazon (128)	1188 MB	703 MB	N/A	1351 MB	704 MB	N/A	1515 MB	705 MB	N/A



- 특히 VRGCN은 층이 깊어질수록 이웃을 더 올려야 해 메모리가 크게 증가합니다.
- GraphSAGE는 높은 수준(약 1.1GB)에서 거의 일정, Cluster-GCN은 약 300MB로 최저·일정.
- Cluster-GCN은 정해진 클러스터 노드만 올려 두고 재사용 → 층이 늘어도 변화 거의 없음.
- 예) Reddit 4-layer : VRGCN 2,064MB ↔ Cluster-GCN 308MB.

결과 ② 계산 시간 : 선형 증가 vs 지수적 폭증

	2-layer	3-layer	4-layer	5-layer	6-layer
Cluster-GCN	52.9s	82.5s	109.4s	137.8s	157.3s
VRGCN	103.6s	229.0s	521.2s	1054s	1956s



- 기존 방법은 층을 쌓을수록 이웃의 이웃을 지수적으로 불러와 계산량이 폭증.
- Cluster-GCN은 클러스터 내부 정보를 한 번 올려 두고 재사용 → 층 수에 선형적으로만 증가.
- 예: 6-layer에서 VRGCN 1,956초 vs Cluster-GCN 157초.

클러스터링, 전처리 시간과 학습 시간

Table 8: Comparisons of running time, memory and testing accuracy (F1 score) for Amazon2M.

	Time		Memory		Test F1 score	
	VRGCN	Cluster-GCN	VRGCN	Cluster-GCN	VRGCN	Cluster-GCN
Amazon2M (2-layer)	337s	1223s	7476 MB	2228 MB	89.03	89.00
Amazon2M (3-layer)	1961s	1523s	11218 MB	2235 MB	90.21	90.21
Amazon2M (4-layer)	N/A	2289s	OOM	2241 MB	N/A	90.41

Table 13: The running time of graph clustering algorithm (METIS) and data preprocessing before the training of GCN.

Datasets	#Partitions	Clustering	Preprocessing
PPI	50	1.6s	20.3s
Reddit	1500	33s	286s
Amazon	200	0.3s	67.5s
Amazon2M	15000	148s	2160s

결과 ③ 정확도 : 기존 방법 대비 SOTA 달성

Table 10: State-of-the-art performance of testing accuracy reported in recent papers.

	PPI	Reddit
FastGCN [1]	N/A	93.7
GraphSAGE [5]	61.2	95.4
VR-GCN [2]	97.8	96.3
GaAN [16]	98.71	96.36
GAT [14]	97.3	N/A
GeniePath [10]	98.5	N/A
Cluster-GCN	99.36	96.60

- PPI에서 F1 99.36, Reddit에서 F1 96.60으로 기존 방법들을 능가. 각각 레이어 5층, 4층을 쌓음
- Cluster GCN은 성능에서도 우수함을 실험적으로 입증

오버 스무딩(Oversmoothing)



수호
캐나다인



규원
펍제라 없앴



인혁
잠시 탈주함



주희
펍제라 없앴2



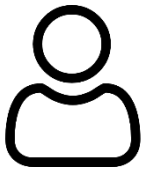
세준
호날두 현 주인



훈의
깃헙 관리자



아카
한국인



은재
술에 약함



연곤
스폰지밥 좋아함



수용
노트북 관리자



승종
호날두 차기 주인

오버 스무딩(Oversmoothing)



수호
중앙대생



규원
중앙대생



인혁
중앙대생



주희
중앙대생



세준
중앙대생



훈의
중앙대생



아카
중앙대생



은재
중앙대생



연곤
중앙대생



수용
중앙대생



승종
중앙대생

Training Deeper GCN

$$X^{(l+1)} = \sigma(A'X^{(l)}W^{(l)})$$

원래 수식 – 이웃 정보(A)를 가져옴

$$X^{(l+1)} = \sigma((A' + I)X^{(l)}W^{(l)})$$

내 정보(I)를 추가 – 이웃정보에 내 정보 추가

$$\tilde{A} = (D + I)^{-1}(A + I),$$

이웃이 너무 많으면 내 정보(I)가 묻히고
너무 적으면 내 정보(I)가 너무 강해져서 정규화

$$X^{(l+1)} = \sigma((\tilde{A} + \lambda \text{diag}(\tilde{A}))X^{(l)}W^{(l)}).$$

내 정보(I)의 영향력을 조절할 수 있도록 가중치(λ)를 곱함

Training Deeper GCN 실험결과

$$Z^{(l+1)} = A'X^{(l)}W^{(l)}, \quad X^{(l+1)} = \sigma(Z^{(l+1)}), \quad (1)$$

$$X^{(l+1)} = \sigma((A' + I)X^{(l)}W^{(l)}) \quad (9)$$

$$\tilde{A} = (D + I)^{-1}(A + I), \quad (10)$$

$$X^{(l+1)} = \sigma((\tilde{A} + \lambda \text{diag}(\tilde{A}))X^{(l)}W^{(l)}). \quad (11)$$

	2-layer	3-layer	4-layer	5-layer	6-layer	7-layer	8-layer
Cluster-GCN with (1)	90.3	97.6	98.2	98.3	94.1	65.4	43.1
Cluster-GCN with (10)	90.2	97.7	98.1	98.4	42.4	42.4	42.4
Cluster-GCN with (10) + (9)	84.9	96.0	97.1	97.6	97.3	43.9	43.8
Cluster-GCN with (10) + (11), $\lambda = 1$	89.6	97.5	98.2	98.3	98.0	97.4	96.2

결론

- Mini-batch SGD는 Full-batch GCN보다 연산량이 급증해 에포크당 학습 시간이 크게 늘어난다.
- 원인은 이전 레이어에서 계산한 임베딩이 다음 레이어에서 거의 재사용되지 않는 것. 즉, embedding utilization이 낮기 때문이다.
- 이를 높이려면 한 배치의 시드 노드들이 이웃을 많이 공유해야 하므로, METIS로 그래프를 클러스터링하고 하나의 클러스터 안에서 배치를 구성한다.
- 단일 클러스터 배치는 편향으로 분산이 커지므로, 여러 클러스터를 묶어 하나의 배치로 학습한다 (Stochastic Multi-Clustering).
- 그 결과 속도·확장성·정확도 모든 면에서 기존 방법을 능가했다.